Initial prompt: Please provide a plan which you suggest, for coding up these ideas, so that you and I can work on refining it prior to you beginning to write python code. If GPU acceleration, we will be using the JAX libraries, not pytorch. Thank you.

Uploaded document was this one: https://github.com/charliederr/causal-hnet/blob/experimental/substitution-prediction/docs/parsing_approach_summary.pdf

This is a fascinating algorithmic challenge. The core intuition—defining syntactic units by the "volume" of their bidirectional substitutability—is powerful, but as the document notes, the primary bottleneck is **scalability** (comparing sets is expensive) and **representation** (how to capture context patterns).

Since you requested **JAX** for acceleration, we should move away from the explicit string-set operations (like Jaccard on Python sets) described in the "Prototype" versions and instead frame the expansion problem as **vectorized similarity operations**. JAX excels at massive matrix multiplications and parallel reductions, which can solve the $O(n^2)$ context comparison bottleneck.

Here is a proposed implementation plan, structured to transition the logic from "symbolic set operations" to "dense vector operations."

Project Plan: JAX-Accelerated Bidirectional Parsing

Phase 1: Data Representation & "The Catalog"

The paper describes a "Catalog" of 32k n-grams with contexts. To make this GPU-compatible, we must vectorize the catalog.

- **1.1 Context Vectorization:** Instead of storing contexts as tuples of strings (left, right), we will encode them into fixed-size vectors.
    - **Approach:** Use a lightweight pre-trained embedding (e.g., a small BERT or GloVe) to generate vectors for the left and right context windows.
    - **Representation:** A context c becomes a vector $v_c \in \mathbb{R}^d$.
    - **Benefit:** This solves "Challenge 3: Representation" immediately.
- **1.2 The Interaction Matrix (Sparse to Dense):**
  We need a global map of which **Units** fit in which **Contexts**.

    - Construct a binary (or frequency-weighted) Adjacency Matrix A:
        - Rows = Unique Units (e.g., "go out", "my money")
        - Cols = Unique Context Clusters (clustered via K-Means to keep dimensions manageable, e.g., 4096 centroids).
    - $A_{ij} = 1$ if Unit i appears in Context Cluster j.
    - **JAX Optimization:** This matrix A will be stored as a JAX array (or jax.experimental.sparse.BCOO) on the GPU.

Phase 2: The Expansion Kernel (JAX Core)

This replaces the iterative Python loops in the bidirectional_expansion function. We will implement the expansion as a "diffusion" or "similarity propagation" step on the GPU.

- **2.1 Context Expansion (The "Where else?"):**
    - **Input:** Vector of the current context v_{ctx}.
    - **Operation:** Compute Cosine Similarity between v_{ctx} and all known Context Centroids (matrix multiplication).
    - **Output:** A "soft mask" vector m_{ctx} (size $N_{clusters}$) where values are similarity scores. This replaces the hard set find_similar_contexts.

- **2.2 Unit Expansion (The "Who else?"):**
    - **Operation:** Multiply the context mask m_{ctx} by the Adjacency Matrix A.
    - **Math:** $v_{units} = A \cdot m_{ctx}$
    - **Result:** A vector where high values indicate units that fit the expanded contexts. This replaces find_units_in_context.

- **2.3 Energy Calculation:**
    - Implement the scoring formula $E = -\log(|units| \times \log(|contexts| + 1))$ using differentiable JAX operations (Softmax/LogSumExp) rather than hard counts. This allows gradients to flow if we later decide to learn the embeddings.

Phase 3: The Parsing Shell (Hybrid CPU/GPU)

The recursive parsing logic (splitting spans) is sequential and branching, which can be tricky in pure JAX. We will use a hybrid approach.

- **3.1 Top-Down Controller (Python):**
    - Manages the recursive splitting of the sentence parse_topdown.

    - Identifies candidate spans (e.g., "my money").
- **3.2 Batch Scoring (JAX):**
    - Instead of scoring one split at a time, we collect *all* candidate spans for a sentence and send them to the GPU in one batch.

    - The JAX kernel returns the "Expansion Energy" for every possible span simultaneously.
- **3.3 Ambiguity Resolution:**
    - Implement the logic to compare unit_energy vs. best_split_energy.

Phase 4: Addressing Specific Challenges

- **Polysemy (Social vs. Motion "go out"):**
    - The plan addresses the "Averaging Problem" by preserving the *input* context vector. Even if the unit "go out" is the same, the *input context* "Did you ___ with Sarah" vs "I ___ the door" will yield different similarity masks m_{ctx} in Step 2.1, activating different parts of the Adjacency Matrix.

- **Scalability:**
    - By using JAX matrix ops, we avoid the $O(n^2)$ comparisons mentioned in the paper. We only compare the query vector against fixed centroids.

Suggested Module Structure

1. data_loader.py:
    - Ingests text.
    - Builds the Unit-Context co-occurrence counts.
    - (Optional) Runs K-Means to compress millions of raw contexts into K centroids.
2. expansion_engine.py (JAX):
    - @jax.jit functions for similarity computation.
    - Holds the large static matrices (Adjacency Matrix A, Context Centroids C) in GPU memory.
3. parser.py:
    - Implements the recursive logic.
    - Batches spans for the engine.

Next Step

Would you like me to begin by detailing the **JAX data structures** for the "Context Centroids" and "Adjacency Matrix," or would you prefer I start by mocking up the **Expansion Engine** code to demonstrate the vector math?

JAX Bidirectional Parsing Plan

Phase 1: Data Representation & "The Catalog"

The paper describes a "Catalog" of 32k n-grams with contexts. To make this GPU-compatible, we must vectorize the catalog.

- **1.1 Context Vectorization:** Instead of storing contexts as tuples of strings `(left, right)`, we will encode them into fixed-size vectors.
  - **Approach:** Use a lightweight pre-trained embedding (e.g., a small BERT or GloVe) to generate vectors for the left and right context windows.
  - **Representation:** A context c becomes a vector $v_c \in \mathbb{R}^d$.
  - **Benefit:** This solves "Challenge 3: Representation" immediately.
- **1.2 The Interaction Matrix (Sparse to Dense):** We need a global map of which **Units** fit in which **Contexts**.
  - Construct a binary (or frequency-weighted) Adjacency Matrix A:
    - Rows = Unique Units (e.g., "go out", "my money")
    - Cols = Unique Context Clusters (clustered via K-Means to keep dimensions manageable, e.g., 4096 centroids).
  - $A_{ij} = 1$ if Unit i appears in Context Cluster j.
  - **JAX Optimization:** This matrix A will be stored as a JAX array (or `jax.experimental.sparse.BCOO`) on the GPU.

Phase 2: The Expansion Kernel (JAX Core)

This replaces the iterative Python loops in the `bidirectional_expansion` function. We will implement the expansion as a "diffusion" or "similarity propagation" step on the GPU.

- **2.1 Context Expansion (The "Where else?"):**
  - **Input:** Vector of the current context v_{ctx}.
  - **Operation:** Compute Cosine Similarity between v_{ctx} and all known Context Centroids (matrix multiplication).
  - **Output:** A "soft mask" vector m_{ctx} (size $N_{clusters}$) where values are similarity scores. This replaces the hard set `find_similar_contexts`.

  - **Output:** A "soft mask" vector m_{ctx} (size $N_{clusters}$) where values are similarity scores. This replaces the hard set `find_similar_contexts`.
- **2.2 Unit Expansion (The "Who else?"):**
  - **Operation:** Multiply the context mask m_{ctx} by the Adjacency Matrix A.
  - **Math:** $v_{units} = A \cdot m_{ctx}$
  - **Result:** A vector where high values indicate units that fit the expanded contexts. This replaces `find_units_in_context`.
- **2.3 Energy Calculation:**
  - Implement the scoring formula $E = -\log(|units| \times \log(|contexts| + 1))$ using differentiable JAX operations (Softmax/LogSumExp) rather than hard counts. This allows gradients to flow if we later decide to learn the embeddings.

Phase 4: Addressing Specific Challenges

- **Polysemy (Social vs. Motion "go out"):**
  - The plan addresses the "Averaging Problem" by preserving the *input* context vector. Even if the unit "go out" is the same, the *input context* "Did you ___ with Sarah" vs "I ___ the door" will yield different similarity masks m_{ctx} in Step 2.1, activating different parts of the Adjacency Matrix.
- **Scalability:**
  - By using JAX matrix ops, we avoid the $O(n^2)$ comparisons mentioned in the paper. We only compare the query vector against fixed centroids.

Suggested Module Structure

1. `data_loader.py`:
   - Ingests text.
   - Builds the Unit-Context co-occurrence counts.
   - (Optional) Runs K-Means to compress millions of raw contexts into K centroids.
2. `expansion_engine.py` (JAX):
   - `@jax.jit` functions for similarity computation.
   - Holds the large static matrices (Adjacency Matrix A, Context Centroids C) in GPU memory.
3. `parser.py`:
   - Implements the recursive logic.
   - Batches spans for the engine.

Next Step

Would you like me to begin by detailing the **JAX data structures** for the "Context Centroids" and "Adjacency Matrix," or would you prefer I start by mocking up the **Expansion Engine** code to demonstrate the vector math?

Sources